PART C

INHERITANCE, INTERFACES POLYMORPHISM

YOUR LEARNING OBJECTIVES

- Inheritance models a substitutable is-a relationship.
- Overload resolution uses compile-time types; overrides dispatch at runtime.
- Interfaces separate contracts from concrete implementations.
- Casting changes the view of a reference, never the object.
- Composition and constructor injection prepare you for Spring Boot.

22 focused concept lessons • one infographic

Mini exercises and reasoned answers included

Understand which calls compile and which implementation actually runs.

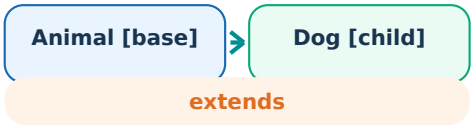
Inheritance models an is-a relationship

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A subclass extends one class and inherits accessible members while adding or refining behavior. The relationship should be substitutable, not merely code reuse.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Animal
{
    String speak()
    {
        return "sound";
    }
}
class Dog extends Animal
{
    @Override
    String speak()
    {
        return "woof";
    }
}
```

EXPECTED OUTPUT

```
No console output (class declarations).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Child extends Base and becomes a subtype of Base.
2. Child inherits accessible Base behavior and may define more behavior.
3. Code expecting Base may accept Child when the is-a relationship is valid.

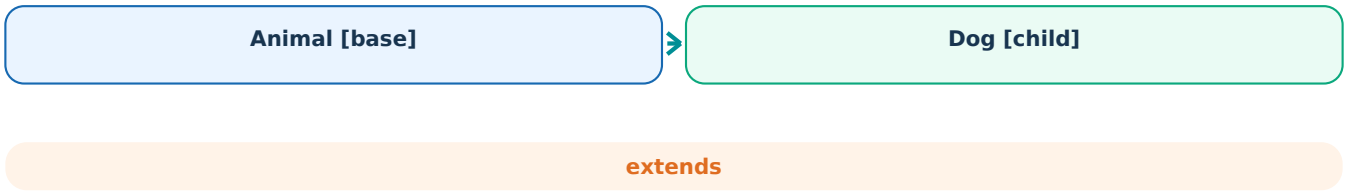
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Inheritance models an is-a relationship

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Child extends Base and becomes a subtype of Base. Child inherits accessible Base behavior and may define more behavior. Code expecting Base may accept Child when the is-a relationship is valid.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Inheritance transmits an accessible interface and behavior, but it creates coupling: a change in the base class can affect all children. The Liskov substitution idea is more important than sharing code: any promised behavior of Animal must still be meaningful for Dog. Prefer a small stable base abstraction over a deep hierarchy with many special-case overrides. Inheritance is strongest when clients can substitute the child wherever the parent is expected without surprising behavior. A subclass does not inherit private member access as callable methods/fields, though the superclass portion of the object still exists. Java permits one direct class superclass, but a class can implement multiple interfaces.

COMMON TRAP

If the subtype cannot preserve the promised behavior of the parent, inheritance may be the wrong design.

TRY THIS IN INTELLIJ

Would Dog extends String compile? Explain final classes.

REMEMBER Code expecting Base may accept Child when the is-a relationship is valid.

APPLY THE RULE / CHANGE ONE THING

Ask whether a Dog is an Animal and whether a Dog is a Collar.

Answer: Dog is an Animal; Dog has a Collar. The second relationship favors composition.

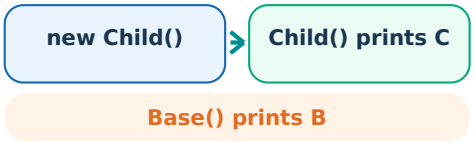
super(...) and constructor execution

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A subclass constructor must initialize its superclass portion before executing its own body. super(...) selects a superclass constructor. Java inserts a no-argument super() when the constructor has no explicit this(...) or super(...) and the superclass supports it.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Base
{
    Base()
    {
        System.out.print("B");
    }
}
class Child extends Base
{
    Child()
    {
        super();
        System.out.print("C");
    }
}
new Child();
```

EXPECTED OUTPUT

```
BC
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new Child() allocates one Child object.
2. The Child constructor invokes Base() first.
3. Base prints B, then Child prints C, giving BC.

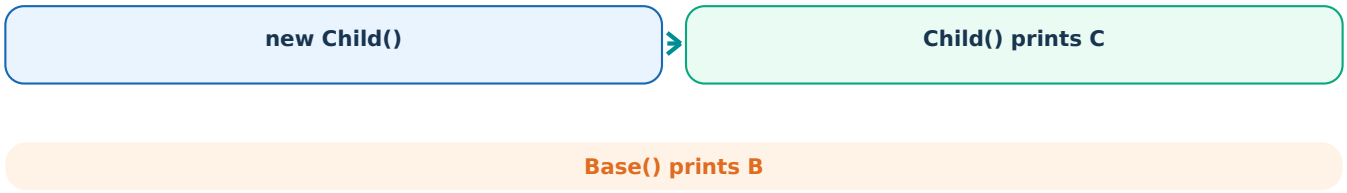
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

super(...) and constructor execution

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

new Child() allocates one Child object. The Child constructor invokes Base() first. Base prints B, then Child prints C, giving BC.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The order starts in Object and works outward through superclasses. The compiler inserts a super() only when appropriate, so a base constructor change can break subclasses even if they never directly use the changed field. Fields are initialized in their own phase, not all at once. Keep constructor logic simple and avoid calling overridable methods from a base constructor. Superclass constructors are invoked in a chain up to Object before subclass constructor bodies run. If no explicit super(...) or this(...) appears, the compiler inserts an implicit no-argument super() where permitted. If the parent has no accessible no-argument constructor, the subclass must select another parent constructor explicitly.

COMMON TRAP

If Base has only Base(int), Child must explicitly invoke an applicable constructor.

TRY THIS IN INTELLIJ

Add a print in an instance initializer and predict the order.

REMEMBER Base prints B, then Child prints C, giving BC.

APPLY THE RULE / CHANGE ONE THING

Remove super() when Base has only Base(int).

Answer: Compilation fails until Child calls an accessible Base(int) constructor.

Overriding gives a new implementation

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An overriding instance method provides subtype-specific behavior for a compatible inherited method signature. @Override makes the compiler verify your intention. Runtime dispatch chooses the implementation associated with the actual receiver object.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal a = new Dog();
System.out.println(a.speak());
// Dog overrides Animal.speak().
```

EXPECTED OUTPUT

```
woof
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The declared Animal reference can call speak().
2. The actual object is Dog and overrides speak().
3. The overriding Dog implementation produces woof.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The annotation @Override catches accidental name and parameter mismatches at compile time. Inherited concrete instance methods are dynamically dispatchable unless restricted by final or other rules. Runtime dispatch uses the actual receiver type, but only after the compiler has found an applicable method in the declared reference type. Only compatible instance methods are dynamically overridden. Using @Override catches signature mistakes early. A subclass cannot override final methods, and private methods are not overridden because they are not inherited as overridable members. Static methods and fields follow different hiding rules.

COMMON TRAP

Different parameter types generally create an overload rather than an override.

TRY THIS IN INTELLIJ

Spell speak as Speak in Dog; what happens to @Override?

REMEMBER The overriding Dog implementation produces woof.

APPLY THE RULE / CHANGE ONE THING

Remove @Override and misspell speak as speek.

Answer: The typo becomes a new method; @Override would have caught it at compile time.

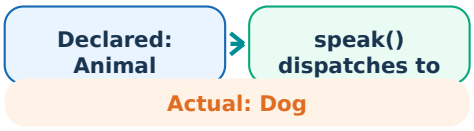
Dynamic dispatch: declared type versus actual object

Understand the rule • trace the code • read the diagram • test it

CORE RULE

The compiler checks accessible members using the declared reference type; the JVM chooses an overridden instance method implementation using the runtime object type. Animal provides the callable contract.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal animal = new Dog("Rex");
System.out.println(animal.speak());
System.out.println(animal.getName());
// animal.fetch(); // not on Animal type
```

EXPECTED OUTPUT

```
woof
Rex
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The compiler checks animal as an Animal reference.
2. It permits speak() and getName() because Animal declares them.
3. At runtime, Dog.speak() executes, while the name comes from the same object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

There are two separate types to track on paper: declared Animal, actual Dog. Declared type answers whether speak() exists on the expression; actual Dog answers which implementation of speak() executes. This distinction also matters when a superclass method itself calls another overridable method: the nested call can dispatch back into the child. Always annotate a reference with two types when tracing: declared type and runtime object type. The declared type controls which methods you may call and can influence overload resolution; the runtime receiver controls the implementation of overridden instance methods. Switching the object to a Cat changes dispatch without changing the client variable type.

COMMON TRAP

The fact that the runtime object is Dog does not allow you to call Dog-only methods through an Animal reference.

TRY THIS IN INTELLIJ

Assign new Cat("Milo") to the same reference and call speak again.

REMEMBER At runtime, Dog.speak() executes, while the name comes from the same object.

APPLY THE RULE / CHANGE ONE THING

Set animal = new Cat("Milo") and call speak().

Answer: The Cat implementation runs while the declared type remains Animal.

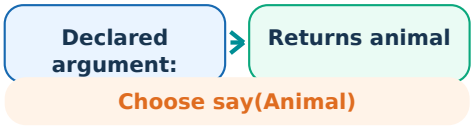
Overloading is selected before dynamic dispatch

Understand the rule • trace the code • read the diagram • test it

CORE RULE

For overloaded methods, Java resolves which signature applies using compile-time types. Only after that does dynamic dispatch select an override for that signature.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static String say(Animal a)
{
    return "animal";
}
static String say(Dog d)
{
    return "dog";
}
Animal ref = new Dog("Rex");
System.out.println(say(ref));
```

EXPECTED OUTPUT

```
animal
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The compiler sees ref has declared type Animal.
2. say(Animal) is selected among overloads at compile time.
3. It returns animal even though ref points at a Dog.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

At this call site, ref has the declared type Animal, so the compiler selects say(Animal). The actual Dog instance does not cause a second overload search. Both say methods in this example are static, so the selected say(Animal) returns "animal" directly. For instance methods, a separate later step could dynamically dispatch an override of the selected signature. Overload choice and override dispatch are separate stages. First determine the signature from compile-time types and conversion rules; only then, if that selected method is overridable, determine the implementation from the receiver’s runtime type. Treating every method choice as “runtime polymorphism” gives wrong answers.

COMMON TRAP

Do not summarize Java dispatch as “the most specific method is always chosen at runtime.”

TRY THIS IN INTELLIJ

Cast ref to Dog in the call and predict the chosen overload.

REMEMBER It returns animal even though ref points at a Dog.

APPLY THE RULE / CHANGE ONE THING

Change only the variable declaration to Dog ref = new Dog(...).

Answer: The call now selects say(Dog) and prints dog.

Static methods are hidden, not overridden

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Static method calls are resolved from compile-time type information. A same-signature static declaration in a subclass hides the inherited static method.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Parent
{
    static String tag()
    {
        return "P";
    }
}
class Child extends Parent
{
    static String tag()
    {
        return "C";
    }
}
Parent p = new Child();
System.out.println(p.tag());
```

EXPECTED OUTPUT

```
P
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Parent p references a Child instance.
2. Static tag() is resolved using Parent at compile time.
3. Parent.tag() returns P, not Child.tag().

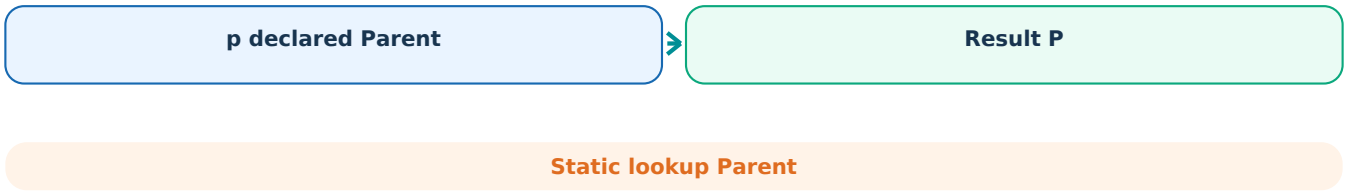
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Static methods are hidden, not overridden

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Parent p references a Child instance. Static tag() is resolved using Parent at compile time. Parent.tag() returns P, not Child.tag().

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Calling p.tag() is legal syntax but misleading for a static method. The compile-time type Parent chooses Parent.tag; there is no virtual dispatch for static functions. Prefer Parent.tag() and Child.tag() in production code. Likewise, a subclass cannot override a static parent declaration with an instance method of the same signature. A same-signature static method declared in a subclass hides a superclass method, rather than overriding it. The recommended call form is Parent.tag() or Child.tag(), not p.tag(), because the latter disguises static selection as an instance operation. The object's runtime subtype does not choose the method.

COMMON TRAP

Do not put @Override on a static method hiding declaration.

TRY THIS IN INTELLIJ

Change p.tag() to Child.tag() and predict the result.

REMEMBER Parent.tag() returns P, not Child.tag().

APPLY THE RULE / CHANGE ONE THING

Replace p.tag() with Child.tag().

Answer: The result changes to C because the class name now selects Child.

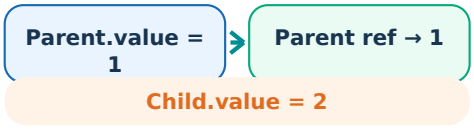
Fields are hidden, not polymorphically overridden

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A field with the same name in a subclass hides the inherited field. The selected field depends on the compile-time reference type.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Parent
{
    int value = 1;
}
class Child extends Parent
{
    int value = 2;
}
Parent p = new Child();
System.out.println(p.value);
```

EXPECTED OUTPUT

```
1
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. p is declared Parent and holds a Child object.
2. p.value is a field access, bound to Parent.value.
3. It prints 1 rather than the Child field value 2.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Two fields with the same simple name can coexist in a child object, one from Parent and another declared in Child. Which one you access depends on the reference type. Encapsulation discourages this pattern: private fields with behavior methods let a subtype supply implementation without exposing confusing duplicate data slots. Fields do not participate in virtual dispatch. A child field with the same name is a separate member, not a replacement of the superclass field. This is usually confusing enough to avoid: keep fields private and expose a method if dynamic subtype-specific behavior is needed.

COMMON TRAP

Hiding fields creates confusion; prefer distinct names and accessors rather than field shadowing.

TRY THIS IN INTELLIJ

Cast p to Child before .value, then predict the result.

REMEMBER It prints 1 rather than the Child field value 2.

APPLY THE RULE / CHANGE ONE THING

Add an overridable getValue() and call p.getValue().

Answer: With a Child override, the method can return 2 via dynamic dispatch.

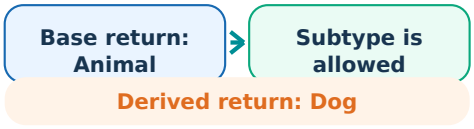
Covariant return types

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An overriding method may return a subtype of the original reference return type (covariant return). Primitive return types cannot change this way.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class AnimalFactory
{
    Animal make()
    {
        return new Dog("Rex");
    }
}
class DogFactory extends AnimalFactory
{
    @Override
    Dog make()
    {
        return new Dog("Bolt");
    }
}
```

EXPECTED OUTPUT

```
No console output (override example).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. AnimalFactory.make declares Animal as its result.
2. DogFactory overrides make and narrows the result to Dog.
3. The override stays compatible because Dog is a subtype of Animal.

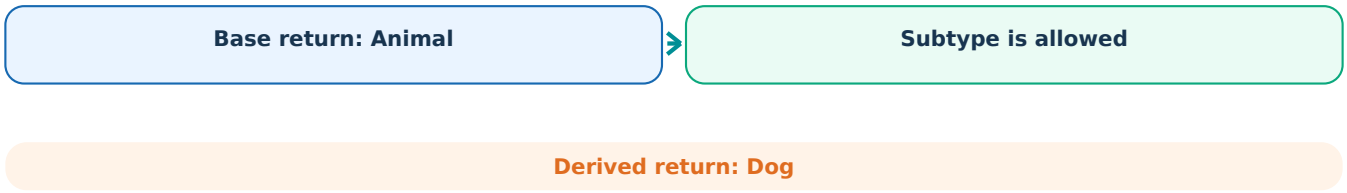
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Covariant return types

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

AnimalFactory.make declares Animal as its result. DogFactory overrides make and narrows the result to Dog. The override stays compatible because Dog is a subtype of Animal.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Covariant return types preserve substitutability while allowing a more specific object result. If AnimalFactory.make() promises Animal, DogFactory.make() may promise Dog. The subtype is still an Animal for callers typed as the base. A changed primitive return type or an unrelated reference return type would break the override contract. Covariant returns help callers of a subtype-specific factory receive more precise return types without casts. They apply to reference returns, not primitive widening. The parameter signature still has to match the overridden method; changing parameters creates an overload instead.

COMMON TRAP

Changing an int-returning method override to long is not a covariant return.

TRY THIS IN INTELLIJ

Change DogFactory.make() to return String and predict compilation.

REMEMBER The override stays compatible because Dog is a subtype of Animal.

APPLY THE RULE / CHANGE ONE THING

Try replacing Dog make() with int make().

Answer: The override is invalid: primitive return changes are not covariant.

Override restrictions: access and exceptions

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An overriding method cannot narrow visibility and cannot declare broader checked exceptions than the overridden contract permits. FileNotFoundException is narrower than IOException; public is more accessible than protected, so this override is legal.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Base
{
    protected void act() throws IOException
    {
    }
}
class Derived extends Base
{
    @Override
    public void act() throws FileNotFoundException
    {
    }
}
```

EXPECTED OUTPUT

```
No console output (legal override).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Base.act is protected and declares IOException.
2. Derived.act is public, increasing visibility.
3. FileNotFoundException is narrower, so the override is legal.

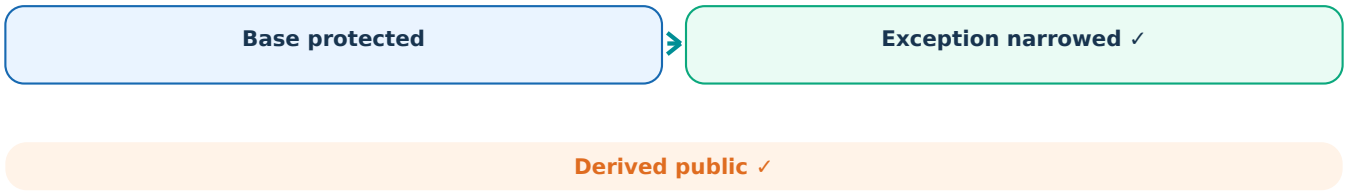
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Override restrictions: access and exceptions

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Base.act is protected and declares IOException. Derived.act is public, increasing visibility. FileNotFoundException is narrower, so the override is legal.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The Java compiler checks both accessibility and checked-exception compatibility. Making an override more public is allowed; narrowing it is not. Throwing FileNotFoundException where IOException was promised is legal, because callers already know how to handle the broader category. Runtime exceptions are handled differently and do not need declaration in throws. An overriding method must preserve the contract callers rely on. It cannot restrict visibility or introduce a broader checked exception; unchecked exceptions follow different rules. Java also checks return compatibility and static/instance consistency. Reading @Override failures is a useful way to learn these contracts.

COMMON TRAP

Trying to make Derived.act private, or to throw Exception in this example, fails compilation.

TRY THIS IN INTELLIJ

Replace FileNotFoundException with Exception. Why is it rejected?

REMEMBER FileNotFoundException is narrower, so the override is legal.

APPLY THE RULE / CHANGE ONE THING

Change Derived.act to private or throws Exception.

Answer: The override fails because visibility is narrower or checked exception is broader.

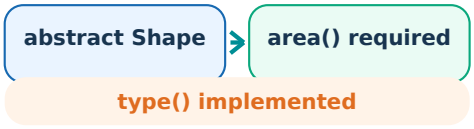
Abstract classes describe partial implementations

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An abstract class cannot be instantiated directly. It may hold constructors, state, implemented methods, and abstract methods requiring subtype implementation.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
abstract class Shape
{
    abstract double area();
    String type()
    {
        return "shape";
    }
}
// new Shape(); // compile error
```

EXPECTED OUTPUT

```
No console output (abstract class).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Shape is abstract and cannot be instantiated directly.
2. It can provide type() behavior while requiring subclasses to implement area().
3. Concrete subclasses become instantiable when all abstract operations are implemented.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An abstract class may carry constructors, state and concrete helper methods, while an interface emphasizes a callable contract. Neither choice is automatically superior. In Java, even an abstract superclass constructor participates in subclass creation. Use abstract classes when shared implementation/state is intrinsic to the subtype relationship, not merely because some methods look similar. An abstract class can have fields, constructors, and implemented methods; it cannot be instantiated directly. It is useful for genuinely shared state and implementation. An interface is usually a better first choice for a capability contract when implementations need unrelated class hierarchies.

COMMON TRAP

Abstract methods have no body and cannot be private, final, or static in a way that would prevent implementation.

TRY THIS IN INTELLIJ

Create Circle extends Shape with area() returning Math.PI*r*r.

REMEMBER Concrete subclasses become instantiable when all abstract operations are implemented.

APPLY THE RULE / CHANGE ONE THING

Make a concrete Rectangle extends Shape without implementing area().

Answer: Compilation fails unless Rectangle itself is declared abstract.

Interfaces define contracts between components

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An interface names operations independent of a concrete implementation. An implementation class promises to provide required instance methods.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
interface PaymentMethod
{
    String pay(int cents);
}
class Card implements PaymentMethod
{
    public String pay(int cents)
    {
        return "CARD:" + cents;
    }
}
```

EXPECTED OUTPUT

```
No console output (contract definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. PaymentMethod declares the operation pay(int).
2. Card implements that contract with a public method.
3. A PaymentMethod reference can later call pay on any implementation.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Interface-typed dependencies make code resistant to concrete implementation changes. A PaymentService can call payment.pay(cents) without knowing whether the actual object is CardPayment or BankTransfer. The implementation method must be public because interface methods declared this way are public. A class can implement many such contracts even though it extends only one class. Interfaces isolate consumers from implementation classes. An implementing method must be public because it fulfills a public interface contract. A class may implement multiple interfaces, and interfaces can extend multiple interfaces; this supports multiple capabilities without multiple class inheritance.

COMMON TRAP

An interface is not a class constructor; new PaymentMethod() is invalid without an anonymous implementation.

TRY THIS IN INTELLIJ

Add another implementation called Bank and run the same client through each.

REMEMBER A PaymentMethod reference can later call pay on any implementation.

APPLY THE RULE / CHANGE ONE THING

Create Cash implements PaymentMethod; pass either Card or Cash to a client.

Answer: The client works through the same PaymentMethod contract.

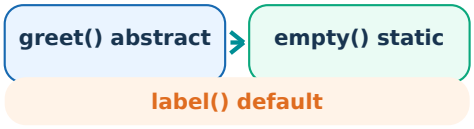
Interface default, static, and private methods

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Modern interfaces can supply default methods, static helper methods, and private helpers used within the interface. Default methods support evolving an interface with behavior; they do not turn an interface into a class with instance fields.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
interface Greeting
{
    String greet();
    default String label()
    {
        return "Greeting";
    }
    static Greeting empty()
    {
        return () -> "";
    }
}
```

EXPECTED OUTPUT

```
No console output (interface definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Greeting declares greet() abstract.
2. label() is a default method available to implementations.
3. Greeting.empty() is an interface static factory returning a lambda.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A default interface method helps add a general behavior while keeping old implementations source-compatible in many cases. Interface static methods are invoked using the interface name. Private interface methods can factor shared logic between defaults or statics. A default method must not be confused with a virtual instance field: interfaces cannot hold mutable per-object state. Default methods offer reusable behavior while allowing old implementations to remain source-compatible when an interface evolves. They are still instance methods. Interface static methods belong to the interface name; private interface methods help share internal default/static implementation logic and are not inherited as public API.

COMMON TRAP

A default method conflict from unrelated interfaces may require an explicit implementation in the class.

TRY THIS IN INTELLIJ

Implement Greeting with an anonymous class, then print label().

REMEMBER Greeting.empty() is an interface static factory returning a lambda.

APPLY THE RULE / CHANGE ONE THING

Attempt to call implementation.label() and Greeting.empty().label().

Answer: Both default calls work when the receiver implements Greeting.

Resolving conflicting default methods

Understand the rule • trace the code • read the diagram • test it

CORE RULE

If two unrelated interfaces provide the same default method, a class implementing both must normally override and resolve the ambiguity. A concrete class can implement tag() itself and may invoke a specific inherited default with Left.super.tag() where permitted.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
interface Left
{
    default String tag()
    {
        return "L";
    }
}
interface Right
{
    default String tag()
    {
        return "R";
    }
}
```

EXPECTED OUTPUT

```
No console output (partial conflict setup).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Left provides default tag() returning L.
2. Right provides default tag() returning R.
3. A class implementing both must explicitly resolve the ambiguous default.

NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Resolving conflicting default methods

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

Left provides default tag() returning L. Right provides default tag() returning R. A class implementing both must explicitly resolve the ambiguous default.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

When both Left and Right provide tag(), Java demands an explicit decision rather than arbitrarily picking one. The implementing class can provide its own tag() and delegate to Left.super.tag() if appropriate. Class methods have priority over interface defaults under Java's resolution rules, which matters in mixed class/interface hierarchies. The class can override tag() with its own body, or choose a specific interface default using Left.super.tag() when the syntax applies. Java also has priority rules: a concrete class method beats an interface default; a more specific subinterface can win over a superinterface default.

COMMON TRAP

Java avoids guessing which unrelated default method the programmer intended.

TRY THIS IN INTELLIJ

Write class Both implements Left, Right and return Left.super.tag() from its override.

REMEMBER A class implementing both must explicitly resolve the ambiguous default.

APPLY THE RULE / CHANGE ONE THING

Implement Left and Right without writing tag().

Answer: Compilation fails due to conflicting inherited defaults.

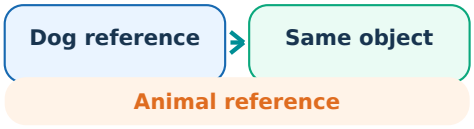
Upcasting to a parent type or interface

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An upcast stores a subtype reference in a variable of a supertype. It is normally implicit and does not change the object.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Dog dog = new Dog("Rex");
Animal animal = dog;
System.out.println(dog == animal);
System.out.println(animal.speak());
```

EXPECTED OUTPUT

```
true
woof
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Dog dog refers to a Dog object.
2. Animal animal = dog copies the reference into a supertype variable.
3. Both references point at the same object; identity comparison prints true.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An upcast does not construct a new object or remove Dog-only fields. It merely changes the reference's declared viewpoint. That means equals, overridden methods, and object identity still observe the same Dog instance. APIs use this to accept a broad Animal or PaymentMethod contract rather than a concrete Dog or CardPayment class. Upcasting changes what the compiler permits through a reference; it does not create or convert an object. This lets clients depend on a stable abstraction instead of concrete implementation details. The object still supports Dog-specific behavior, but reaching it through an Animal reference requires an appropriate narrowing cast.

COMMON TRAP

Upcasting does not slice objects in Java; the whole original object remains in memory.

TRY THIS IN INTELLIJ

Assign a Dog object to an Object variable and call toString().

REMEMBER Both references point at the same object; identity comparison prints true.

APPLY THE RULE / CHANGE ONE THING

Mutate the Dog through dog after assigning animal = dog.

Answer: The Animal reference sees the same object's changed state.

Downcasting: a runtime assertion about type

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A downcast requests a subtype reference. It can compile even when it later fails because the object is not actually an instance of that subtype.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal animal = new Cat("Milo");
try
{
    Dog dog = (Dog) animal;
}
catch (ClassCastException e)
{
    System.out.println("Wrong subtype");
}
```

EXPECTED OUTPUT

```
Wrong subtype
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. animal refers to a Cat object.
2. The cast to Dog compiles because an Animal could be a Dog.
3. At runtime the Cat is incompatible, so ClassCastException is caught.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A downcast is valid only when the runtime object really belongs to the target subtype. The language may reject casts that can never be valid at compile time; otherwise an incompatible runtime object leads to ClassCastException. A design that repeatedly downcasts from interfaces may be missing a necessary operation on the interface itself. A cast asserts a subtype relationship about the existing object. It does not manufacture a different subtype or change the object's runtime class. Prefer instanceof checks or better interface design when casts would otherwise be common. If a cast is impossible by type rules, Java may reject it at compile time.

COMMON TRAP

Casting an unrelated final type may be rejected at compile time rather than reaching ClassCastException.

TRY THIS IN INTELLIJ

Replace new Cat with new Dog and call a Dog-only method.

REMEMBER At runtime the Cat is incompatible, so ClassCastException is caught.

APPLY THE RULE / CHANGE ONE THING

Replace new Cat with new Dog in the setup.

Answer: The Dog cast succeeds and can be used for Dog-only methods.

instanceof and pattern matching (Java 17)

Understand the rule • trace the code • read the diagram • test it

CORE RULE

instanceof tests whether a reference is compatible with a type. Java 17 pattern matching can introduce a typed binding in a successful test.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal animal = new Dog("Rex");
if (animal instanceof Dog dog)
{
    System.out.println(dog.fetch());
}
```

EXPECTED OUTPUT

```
fetching
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. animal points at a Dog.
2. instanceof Dog succeeds and binds dog as a typed pattern variable.
3. dog.fetch() is callable inside the guarded branch and prints fetching.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Pattern matching reduces redundant test-then-cast boilerplate but retains the same underlying type safety. The variable dog is in scope where the compiler knows the match succeeded, including flow-sensitive combinations such as conditions involving &&. Do not assume the variable is available after any arbitrary if; scope depends on the code path analysis. Pattern matching avoids a redundant explicit cast and introduces a scoped binding that exists where the compiler can prove the test succeeded. If the object is null, instanceof returns false. Flow-scoping rules matter when tests are combined with negation and logical operators.

COMMON TRAP

instanceof null is false. It does not need a separate null check to return false.

TRY THIS IN INTELLIJ

Try Animal animal = null and predict whether the body runs.

REMEMBER dog.fetch() is callable inside the guarded branch and prints fetching.

APPLY THE RULE / CHANGE ONE THING

Change animal to null.

Answer: The condition is false and the block prints nothing.

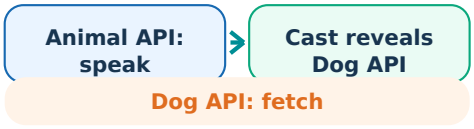
What you can call is a compile-time question

Understand the rule • trace the code • read the diagram • test it

CORE RULE

The declared type determines method availability at the call site, even if the actual object contains additional public methods. Dynamic dispatch answers which implementation of an accessible overridden method runs.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Animal a = new Dog("Rex");
System.out.println(a.speak());
// a.fetch();     // compile error
System.out.println(((Dog) a).fetch());
```

EXPECTED OUTPUT

```
woof
fetching
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The declared type Animal exposes speak but not fetch.
2. The first call dynamically invokes Dog.speak and prints woof.
3. A checked cast exposes fetch and the second call prints fetching.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The declared reference type is a compile-time promise. If an interface does not expose fetch(), callers cannot use it through that interface even if the actual object happens to be a Dog. Casting can expose the extra method, but that couples the caller to an implementation detail. Interfaces should contain the operations clients genuinely need. Compile-time accessibility and runtime dispatch answer different questions. A downcast makes a narrower method set available to the compiler but must still be valid for the runtime object. If a client continually needs Dog-only operations, revisit whether the Animal abstraction has the right responsibilities.

COMMON TRAP

An interface variable cannot call methods missing from its declared interface simply because you know the implementation class.

TRY THIS IN INTELLIJ

Create PaymentMethod method = new CardPayment() and list which methods are callable.

REMEMBER A checked cast exposes fetch and the second call prints fetching.

APPLY THE RULE / CHANGE ONE THING

Comment out the cast and attempt a.fetch().

Answer: Compilation fails because fetch is not declared on Animal.

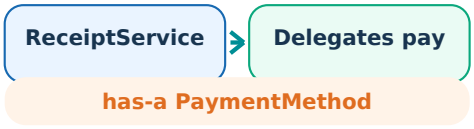
Composition versus inheritance: choosing wisely

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Use inheritance for a genuine substitutable is-a relationship. Use composition when one object should use another object's behavior.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class ReceiptService
{
    private final PaymentMethod payment;
    ReceiptService(PaymentMethod payment)
    {
        this.payment = payment;
    }
    String process(int cents)
    {
        return payment.pay(cents);
    }
}
```

EXPECTED OUTPUT

```
No console output (delegating class).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. ReceiptService stores the supplied PaymentMethod collaborator.
2. process delegates to payment.pay(cents).
3. The service does not need to extend PaymentMethod to use that behavior.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Composition helps avoid brittle base-class assumptions. ReceiptService can delegate to a payment interface and be tested with a fake implementation; it does not need to extend the payment processor. Inheritance remains appropriate for well-defined subtype families, but it is harder to revise because base behavior and constructor rules propagate into every child. Inheritance should be reserved for a true substitutable is-a relationship. Composition connects independent responsibilities and makes collaborators replaceable, testable, and easier to evolve. Prefer depending on an interface through constructor injection, as in this example, so replacing Card with Cash does not require changing ReceiptService.

COMMON TRAP

"Prefer composition" is guidance, not a ban on inheritance; abstract hierarchies still have legitimate uses.

TRY THIS IN INTELLIJ

Swap CardPayment for BankTransfer without changing ReceiptService.

REMEMBER The service does not need to extend PaymentMethod to use that behavior.

APPLY THE RULE / CHANGE ONE THING

Swap Card for a test PaymentMethod returning a fixed value.

Answer: ReceiptService remains unchanged; the dependency supplies different behavior.

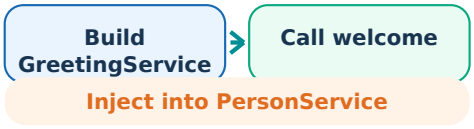
Dependency injection before Spring

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Constructor injection is ordinary Java: a class receives an abstraction from the outside instead of constructing a concrete dependency internally. An interface-typed field and constructor makes a dependency explicit.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
GreetingService greeter =
    new FriendlyGreetingService();
PersonService people =
    new PersonService(greeter);
System.out.println(people.welcome(person));
```

EXPECTED OUTPUT

```
Hello, Asha Rao! (project demo)
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. An implementation of GreetingService is constructed.
2. The PersonService constructor receives that interface reference.
3. welcome delegates to the injected service and prints the supplied greeting.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Constructor injection means the caller chooses the collaborator and supplies it as a dependency. Spring later builds on the same idea by managing bean lifecycles and wiring dependencies. The plain Java example already teaches the important structure: interface-typed field, explicit constructor contract and interchangeable implementations. No framework magic is required to understand it. Dependency injection is a design pattern before it is a framework feature. The caller or composition root decides which implementation to construct; the consumer declares its needs through a constructor. Spring will later automate much of the assembly, but the principles of abstraction, ownership, and testability are ordinary Java.

COMMON TRAP

Creating new FriendlyGreetingService() inside PersonService would hide the collaborator and make substitution harder.

TRY THIS IN INTELLIJ

Replace FriendlyGreetingService with a second implementation.

REMEMBER welcome delegates to the injected service and prints the supplied greeting.

APPLY THE RULE / CHANGE ONE THING

Pass a FakeGreetingService with a predictable output instead.

Answer: PersonService can be tested without changing its implementation.

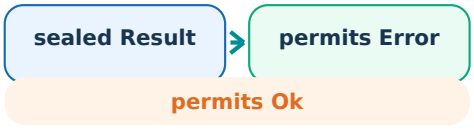
sealed hierarchies in Java 17

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A sealed class or interface restricts which directly declared types may extend or implement it. Java 17 supports sealed, permits, non-sealed, and final.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
sealed interface Result
    permits Ok, Error
{
}
final class Ok implements Result
{
}
final class Error implements Result
{
}
```

EXPECTED OUTPUT

```
No console output (closed hierarchy).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Result declares the only permitted direct implementations.
2. Ok and Error are final concrete implementations.
3. No unlisted class may directly implement Result in an otherwise valid context.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Sealed classes and interfaces are a Java 17 language feature. Direct permitted subtypes must obey the sealed/final/non-sealed classification, so the type family's expansion is controlled. This helps when reasoning about a limited set of outcomes. Sealed types do not eliminate the need to design meaningful contracts or handle unknown states when integrating external input. Sealed hierarchies make allowed subtype sets explicit. Permitted direct subclasses must themselves be final, sealed, or non-sealed as required, and permitted types must satisfy language locality rules. In Java 17, sealed types are useful when modeling finite result families, though ordinary interfaces remain better for open extension points.

COMMON TRAP

A sealed interface is not automatically an enum and does not force all implementations into one source file.

TRY THIS IN INTELLIJ

Try adding class Pending implements Result without changing permits.

REMEMBER No unlisted class may directly implement Result in an otherwise valid context.

APPLY THE RULE / CHANGE ONE THING

Add class Pending implements Result without permitting it.

Answer: The new direct implementation is rejected by the compiler.

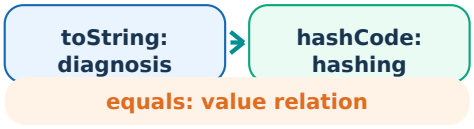
Object methods: toString, equals, hashCode

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Every class inherits core Object behavior. toString supports a readable diagnostic representation; equals/hashCode define equality and hashing semantics. Override Object methods together where relevant and keep their contracts consistent.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
Person p = new Person(101, "Asha");
System.out.println(p.toString());
System.out.println(p.equals(p));
```

EXPECTED OUTPUT

```
Person[id=101, name=Asha] (illustrative format)
true
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The created Person has a readable toString() in this illustration.
2. p.equals(p) is true by reflexivity.
3. If logical equality is defined, hashCode must be consistent with it.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

toString is primarily for debugging and diagnostics; do not rely on its default format for serialization. equals and hashCode should describe stable logical identity where needed. getClass reports the actual runtime class; casts do not alter it. Sensitive values like passwords or tokens should not leak into automatically generated toString methods. Object supplies foundational methods, but their default implementations may not match your domain’s identity requirements. If you override equals(), also override hashCode(): equal objects must have equal hashes. A custom toString() should support diagnosis without exposing secrets. Records synthesize value-oriented implementations for their components.

COMMON TRAP

A stable toString format is not guaranteed by default Object.toString and should not be parsed as business data.

TRY THIS IN INTELLIJ

Compare p.toString() with the overridden output in the companion project.

REMEMBER If logical equality is defined, hashCode must be consistent with it.

APPLY THE RULE / CHANGE ONE THING

Insert two equal value objects into a HashSet without a consistent hashCode().

Answer: Hash-based collections can behave incorrectly when equals/hashCode contracts are broken.

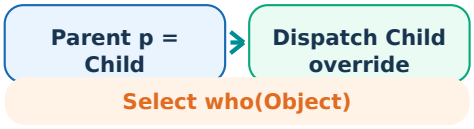
Trace declared types, overloads, and overrides

Understand the rule • trace the code • read the diagram • test it

CORE RULE

When a complex expression combines inheritance with overloaded methods, resolve signatures first, then find runtime implementation. Compiler sees Parent and chooses who(Object).

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Parent
{
    String who(Object o)
    {
        return "P:Object";
    }
}
class Child extends Parent
{
    @Override
    String who(Object o)
    {
        return "C:Object";
    }
    String who(String s)
    {
        return "C:String";
    }
}
Parent p = new Child();
System.out.println(p.who("hi"));
```

EXPECTED OUTPUT

```
C:Object
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The variable p is declared Parent though it holds a Child.
2. At compile time, only who(Object) is visible on Parent.
3. At runtime, Child overrides who(Object), so it prints C:Object.

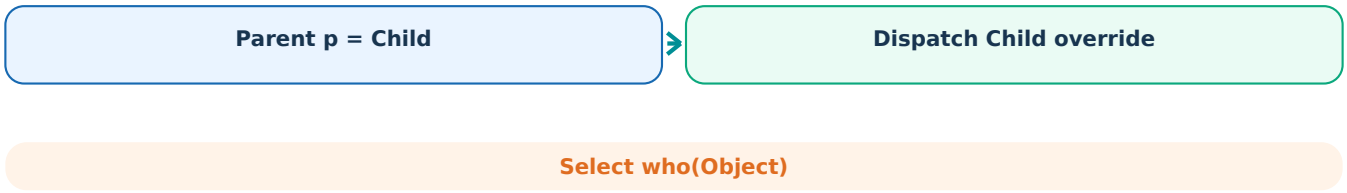
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Trace declared types, overloads, and overrides

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

The variable p is declared Parent though it holds a Child. At compile time, only who(Object) is visible on Parent. At runtime, Child overrides who(Object), so it prints C:Object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The reliable algorithm is: (1) list candidates visible through the declared receiver type, (2) resolve overload using compile-time argument types, (3) choose the runtime override for that selected signature. Child.who(String) is not considered through a Parent variable because the Parent API does not declare it.

COMMON TRAP

Do not skip the compile-time resolution step just because the runtime object is a subclass.

TRY THIS IN INTELLIJ

Change the declared type of p to Child and predict the result.

REMEMBER At runtime, Child overrides who(Object), so it prints C:Object.

APPLY THE RULE / CHANGE ONE THING

Change Parent p to Child p without changing new Child().

Answer: Now who(String) can be selected, producing C:String.

CHAPTER 3 · PART C · VISUAL MASTER MAP

Core rules • mental models • common pitfalls • executable examples



01 INHERITANCE

- extends means substitutable is-a.
- Superclass construction happens first.
- Do not inherit only to reuse code.

02 OVERRIDING

- Same compatible instance signature.
- Runtime receiver chooses implementation.
- Use @Override to catch mistakes.

03 INTERFACES

- Contracts separate callers from implementations.
- One class implements many interfaces.
- Default methods may provide shared behavior.

04 OVERLOADING

- Different parameter lists.
- Chosen with compile-time argument types.
- Not itself runtime polymorphism.

05 CASTING & TYPE TESTS

- Upcasting is normally implicit.
- Downcast can fail at runtime.
- instanceof patterns narrow safely.

06 COMPOSITION & DI

- Has-a instead of fragile is-a.
- Constructor-inject interfaces.
- Spring later automates this wiring.

EXAMPLE — SOURCE AND OUTPUT

CODE / RULE

```
Animal a = new Dog("Rex");
System.out.println(a.speak());
// Dog.speak() executes at runtime.
```

EXPECTED RESULT

```
woof
```

PART C / PRACTICE

Predict before running

9 thinking questions. Answer from the Java rule, not from memorized output.

C1 Why does Animal a = new Dog() allow a.speak() but not a.fetch()?

C2 When a call is overloaded and overridden, which decision happens first?

C3 Is a same-named static method in a subclass an override?

C4 Does a field access dynamically dispatch like a method call?

C5 Why is a Cat-to-Dog downcast unsafe?

C6 What does instanceof Dog dog introduce?

C7 When are sealed hierarchies helpful?

C8 Why prefer composition for ReceiptService and PaymentMethod?

C9 What is dependency injection without Spring?

PART C / ANSWER KEY

Reasoning and answers

Explain each rule in your own words before moving on.

C1 Member lookup uses declared Animal type; overriding of speak dispatches using the Dog runtime receiver.

C2 Compiler selects a method signature from declared types; runtime dispatch chooses an override of that signature.

C3 No. Static methods are hidden and selected via compile-time type.

C4 No. Fields are selected using compile-time reference type.

C5 A cast does not transform the object; incompatible casts throw ClassCastException (or fail compilation).

C6 A pattern variable of Dog type available where a successful type test is known.

C7 When the direct subtype family should be explicitly constrained and understood.

C8 ReceiptService uses payment behavior; it is not a kind of payment implementation.

C9 Construct a class with collaborators supplied from outside, often as interface-typed constructor parameters.